

Da arquitetura cliente-servidor à primeira aplicação web

HTML, CSS e JavaScript em um exemplo integrado



Prof. Dr. João Choma

Universidade Estadual de Maringá — DIN/CTC

github.com/JoaoChoma

Programação de Sistemas Web — Semana 02

Da arquitetura ao código



Nesta aula, construiremos um modelo mental da Web e o aplicaremos em uma pequena interface de pedido de pizza.



A pergunta que guiará a aula

Ideia central

O que acontece entre o clique do usuário e o resultado exibido na tela?

Responderemos acompanhando os dados por quatro pontos:

- 1 interface;
- 2 navegador;
- 3 rede;
- 4 servidor.



Ao final, você deverá ser capaz de

- explicar os papéis de cliente, servidor e rede;
- descrever o ciclo de requisição e resposta;
- diferenciar apresentação, regras de negócio e dados;
- usar HTML para estruturar conteúdo;
- usar CSS para apresentar o conteúdo;
- usar JavaScript para reagir às ações do usuário;
- construir e testar uma pequena aplicação no navegador.

Parte 1 — A Web é uma conversa



O exemplo começa com uma ação cotidiana

Uma pessoa abre o site de uma pizzeria, escolhe uma pizza e confirma o pedido. Parece uma única ação, mas vários componentes colaboram para realizá-la.

Pergunta: quais tarefas pertencem ao dispositivo do cliente e quais dependem da pizzeria?



- **Cliente:** inicia a interação e apresenta a interface.
- **Servidor:** recebe solicitações e oferece serviços ou recursos.
- **Rede:** transporta as mensagens entre os dois lados.

Na Web, o navegador normalmente executa o papel de cliente.



Um celular pode ser cliente de um sistema e servidor em outro contexto.
O papel depende da interação:

- quem **solicita** um serviço atua como cliente;
- quem **oferece** o serviço atua como servidor.

Uma mesma máquina pode executar processos com papéis diferentes.



Responsabilidades frequentes do cliente:

- apresentar informações;
- coletar dados;
- realizar validações imediatas;
- reagir a cliques e digitação;
- enviar solicitações;
- apresentar respostas e erros.

O cliente melhora a interação, mas não deve ser a única proteção do sistema.



Responsabilidades frequentes do servidor:

- autenticar e autorizar usuários;
- aplicar regras de negócio;
- coordenar operações concorrentes;
- acessar dados persistentes;
- integrar outros serviços;
- produzir respostas consistentes.

O servidor atende muitos clientes e não pode confiar cegamente nos dados recebidos.



A rede leva bytes de um ponto a outro.

Protocolos atribuem significado à troca:

- **Ethernet/Wi-Fi:** acesso ao meio físico;
- **IP:** endereçamento e encaminhamento;
- **TCP:** transporte confiável e ordenado;
- **HTTP:** formato e significado da conversa web.

Cada camada resolve uma parte diferente do problema.



Exemplo simplificado:

```
GET /cardapio HTTP/1.1  
Host: pizzaria.exemplo  
Accept: text/html
```

O cliente informa uma operação, um recurso e metadados.



```
HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8  
  
<h1>Cardápio</h1>
```

O status 200 indica sucesso; o corpo contém uma representação do recurso.



- 1 O usuário realiza uma ação.
- 2 O cliente constrói uma requisição.
- 3 A rede transporta a mensagem.
- 4 O servidor processa a solicitação.
- 5 O servidor produz uma resposta.
- 6 O cliente interpreta e apresenta o resultado.

Depurar uma aplicação é descobrir em qual passo a expectativa deixou de ser atendida.



```
GET /cardapio HTTP/1.1
```

```
POST /pedidos HTTP/1.1
```

```
Content-Type: application/json
```

```
{"sabor":"calabresa","tamanho":"grande"}
```

O método ajuda a comunicar a intenção da operação.



Status	Significado no exemplo
200	Consulta realizada com sucesso
201	Pedido criado
400	Dados do pedido são inválidos
404	Recurso não foi encontrado
500	O servidor falhou ao processar

Uma resposta útil informa tanto o resultado quanto a natureza de uma falha.



Ao clicar em **Ver cardápio**:

- quem inicia a conversa?
- qual recurso poderia ser solicitado?
- qual método seria apropriado?
- o que indicaria sucesso?
- onde a resposta seria apresentada?

Compare suas respostas com o percurso de seis passos.

Parte 2 — Organizar evita misturar responsabilidades



Três responsabilidades aparecem em quase toda aplicação

- **Apresentação:** interação e visualização.
- **Negócio:** decisões e regras do domínio.
- **Dados:** armazenamento e recuperação persistente.

Essa separação é conceitual. A implantação física pode variar.



No pedido de pizza, a apresentação inclui:

- cardápio;
- campos do formulário;
- botão de confirmação;
- mensagens de validação;
- resumo do preço.

Seu objetivo é tornar o estado e as ações compreensíveis.



Exemplos:

- pizza grande custa mais que pizza pequena;
- retirada não possui taxa de entrega;
- pedido entregue exige endereço;
- cupom vencido não pode gerar desconto;
- pedido confirmado não pode ter total negativo.

Uma regra importante deve ser garantida no servidor, mesmo que o cliente também a verifique.



Ela pode armazenar:

- clientes;
- itens do cardápio;
- preços vigentes;
- pedidos;
- pagamentos;
- histórico de alterações.

Persistência significa que o dado sobrevive ao encerramento da página ou do processo.



Se o preço mudar, a regra de cálculo pode ser alterada sem redesenhar toda a tela.
Se o visual mudar, os pedidos armazenados não precisam mudar.
Se o banco mudar, o contrato da interface pode permanecer estável.
Separação reduz o impacto de cada mudança; não elimina a necessidade de integração.



Em um monólito, várias responsabilidades são implantadas como uma unidade.

Vantagens possíveis:

- desenvolvimento inicial simples;
- execução e depuração locais;
- menor custo operacional.

O problema surge quando responsabilidades ficam misturadas e difíceis de evoluir.



Sistemas distribuídos permitem escalar e implantar partes separadamente, mas introduzem:

- latência;
- falhas parciais;
- concorrência;
- observabilidade;
- consistência entre serviços;
- maior complexidade operacional.

Distribuir é uma decisão de engenharia, não um objetivo por si só.



Em uma arquitetura P2P, participantes podem oferecer e solicitar recursos. Isso contrasta com uma interação cliente-servidor típica, na qual os papéis de uma conversa são claros.

Importante: um participante pode atuar como cliente em uma operação e como servidor em outra.



Onde cada ação deve ocorrer prioritariamente?

- ① Mudar a cor de um botão ao passar o mouse.
- ② Verificar se o campo “nome” está vazio.
- ③ Confirmar se o cupom ainda é válido.
- ④ Salvar o pedido.
- ⑤ Impedir que duas vendas usem o último item disponível.

Justifique usando **apresentação**, **negócio** e **dados**.

Parte 3 — O navegador executa três tecnologias fundamentais



Tecnologia	Pergunta principal
HTML	O que este conteúdo significa?
CSS	Como ele deve ser apresentado?
JavaScript	O que acontece quando o estado muda?

Separar esses papéis torna a página mais legível e fácil de manter.



```
<h1>Pizzaria Web</h1>
<p>Monte seu pedido.</p>

<form id="pedido">
  <label for="sabor">Sabor</label>
  <select id="sabor" name="sabor"></select>
  <button type="submit">Calcular</button>
</form>
```

Os elementos dizem o que cada conteúdo representa.



```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pizzaria Web</title>
</head>
<body>
  <h1>Pizzaria Web</h1>
</body>
</html>
```

head descreve o documento; body contém o conteúdo apresentado.



```
<a href="cardapio.html">Ver cardápio</a>
```

- `<a>` e `</a>` delimitam o elemento;
- `href` indica o destino;
- `Ver cardápio` é o conteúdo do link.

Nem todo elemento possui fechamento, como `img` e `input`.



```
<header>...</header>
<main>
  <section>...</section>
</main>
<footer>...</footer>
```

Elementos semânticos comunicam a função do conteúdo ao navegador, a tecnologias assistivas e a quem mantém o código.



```
<h1>Pizzaria Web</h1>  
<h2>Monte seu pedido</h2>  
<h3>Escolha o tamanho</h3>
```

Use níveis para representar a estrutura, não apenas para obter tamanhos visuais.



```
<ul>
  <li>Calabresa</li>
  <li>Marguerita</li>
  <li>Vegetariana</li>
</ul>
```

Use `ol` quando a ordem dos itens tiver significado.



```
<label for="quantidade">Quantidade</label>  
<input id="quantidade" name="quantidade"  
      type="number" min="1" value="1">
```

O `label` dá nome ao campo; `name` identifica o dado enviado; `type` informa o tipo de entrada.



```
document
|-- main
   |-- h1
   |-- form#pedido
      |-- label
      |-- select#sabor
      |-- button
```

JavaScript consulta e modifica essa árvore; CSS seleciona seus elementos.



```
body {  
  font-family: system-ui, sans-serif;  
  background: #fff7ed;  
  color: #292524;  
}  
  
h1 {  
  color: #b91c1c;  
}
```

Cada regra combina um seletor com propriedades e valores.



```
<p class="mensagem sucesso">Pedido calculado.</p>
```

```
.mensagem { padding: 1rem; }  
.sucesso  { background: #dcfce7; }
```

Uma classe descreve um grupo reutilizável; id identifica um elemento específico.



Todo elemento pode ter:

- ① conteúdo;
- ② preenchimento (padding);
- ③ borda (border);
- ④ margem (margin).

```
* { box-sizing: border-box; }
```

Com border-box, largura e altura incluem preenchimento e borda.



```
main {  
  width: min(42rem, 100% - 2rem);  
  margin: 2rem auto;  
}  
  
input, select, button {  
  width: 100%;  
  padding: .75rem;  
}
```

A interface usa o espaço disponível sem ultrapassar uma largura confortável.



```
const botao = document.querySelector("button");

botao.addEventListener("click", () => {
  console.log("O usuário clicou");
});
```

O navegador chama a função quando o evento acontece.



```
<script src="app.js" defer></script>
```

`defer` permite baixar o script enquanto o HTML é analisado e executá-lo depois que a estrutura do documento estiver disponível.

Separar arquivos favorece organização e reutilização.



```
const preco = 40;  
const Preco = 50;
```

preco e Preco são identificadores diferentes.

Uma convenção comum para variáveis e funções é camelCase:

```
const precoUnitario = 40;
```



```
const sabor = "calabresa";  
let quantidade = 1;  
  
quantidade = quantidade + 1;
```

- `const`: a variável não recebe outra atribuição;
- `let`: o valor poderá ser reatribuído;
- `var`: existe em códigos antigos, mas possui regras de escopo menos intuitivas.



Valores possuem tipos

```
const nome = "Ana";           // string
const quantidade = 2;         // number
const entrega = true;         // boolean
const observacao = null;      // ausência intencional
const sabores = ["queijo"];   // array (objeto)
const pedido = { total: 80 }; // object
```

JavaScript é dinamicamente tipada: o tipo pertence ao valor em tempo de execução.



```
const campo = document.querySelector("#quantidade");  
const quantidade = Number(campo.value);
```

Sem conversão:

```
"2" + "3" // "23"
```

Com conversão:

```
Number("2") + Number("3") // 5
```




```
const subtotal = preco * quantidade;  
const temEntrega = modalidade === "entrega";  
const taxa = temEntrega ? 8 : 0;  
const total = subtotal + taxa;
```

Use `===` para comparar valor e tipo sem conversão implícita.



```
const mensagem = `Total do pedido: R$ ${total.toFixed(2)}`;
```

`toFixed(2)` produz uma string com duas casas decimais.

Para aplicações reais, formatação monetária deve considerar localidade e moeda.



```
function calcularTotal(preco, quantidade, taxa) {  
  return preco * quantidade + taxa;  
}  
  
const total = calcularTotal(40, 2, 8);
```

Entradas explícitas e retorno previsível tornam a função mais fácil de testar.



```
function taxaDeEntrega(modalidade) {  
  if (modalidade === "entrega") {  
    return 8;  
  }  
  
  return 0;  
}
```

Uma função pequena pode representar uma regra do domínio.



```
const formulario = document.querySelector("#pedido");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  console.log("Vamos processar o pedido");
});
```

`preventDefault()` evita o envio tradicional enquanto processamos os dados com JavaScript.



```
const sabor = document.querySelector("#sabor").value;  
const quantidade = Number(  
  document.querySelector("#quantidade").value  
);
```

Ler, validar, calcular e apresentar são etapas diferentes; separá-las facilita a depuração.



```
if (!Number.isInteger(quantidade) || quantidade < 1) {  
    saida.textContent = "Informe uma quantidade inteira a partir de 1.";  
    return;  
}
```

Uma boa mensagem informa o problema e a ação esperada.



```
const saida = document.querySelector("#resultado");  
saida.textContent = `Total: R$ ${total.toFixed(2)}`;
```

Para texto fornecido pelo usuário, prefira textContent a montar HTML com innerHTML.



Use as ferramentas do navegador para observar:

- **Elements:** HTML e estilos aplicados;
- **Console:** mensagens e erros de JavaScript;
- **Network:** requisições e respostas;
- **Sources:** arquivos e pontos de interrupção.

Um erro no console é uma pista, não apenas um obstáculo.



```
const preco = "40";  
const Quantidade = 2;  
  
if (quantidade = 2) {  
    console.log("Total: " + preco + quantidade);  
}
```

Considere:

- nomes de variáveis;
- atribuição versus comparação;
- soma versus concatenação.

Parte 4 — Laboratório: Pedido de Pizza



O usuário deverá conseguir:

- 1 escolher um sabor;
- 2 escolher um tamanho;
- 3 informar uma quantidade;
- 4 escolher retirada ou entrega;
- 5 calcular o total;
- 6 receber uma mensagem clara de validação.

O foco é a integração entre estrutura, apresentação e comportamento.



Escolha	Regra
Pequena	preço-base do sabor
Média	preço-base + R\$ 8,00
Grande	preço-base + R\$ 14,00
Retirada	sem taxa
Entrega	taxa fixa de R\$ 8,00
Quantidade	inteiro entre 1 e 10

O total é $(\text{preço-base} + \text{adicional}) \times \text{quantidade} + \text{taxa}$.



```
pizzaria-web/  
+-- index.html  
+-- estilos.css  
`-- app.js
```

Abra a pasta em um editor e carregue `index.html` no navegador.
Após cada passo, recarregue a página e observe o console.



```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pizzaria Web</title>
  <link rel="stylesheet" href="estilos.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main></main>
</body>
</html>
```

Teste: a guia do navegador deve exibir “Pizzaria Web”.



```
<main>
  <header>
    <p class="marca">Pizzaria Web</p>
    <h1>Monte seu pedido</h1>
    <p>Escolha as opções e veja o total antes de confirmar.</p>
  </header>

  <form id="pedido"></form>
  <output id="resultado" aria-live="polite"></output>
</main>
```

aria-live ajuda tecnologias assistivas a anunciar a mudança do resultado.



```
<label for="sabor">Sabor</label>
<select id="sabor" name="sabor" required>
  <option value="calabresa">Calabresa - R$ 38</option>
  <option value="marguerita">Marguerita - R$ 36</option>
  <option value="vegetariana">Vegetariana - R$ 40</option>
</select>

<label for="tamanho">Tamanho</label>
<select id="tamanho" name="tamanho" required>
  <option value="pequena">Pequena</option>
  <option value="media">Média (+ R$ 8)</option>
  <option value="grande">Grande (+ R$ 14)</option>
</select>
```

Insira este trecho dentro de form.



```
<label for="quantidade">Quantidade</label>
<input id="quantidade" name="quantidade"
      type="number" min="1" max="10" value="1" required>

<label for="modalidade">Recebimento</label>
<select id="modalidade" name="modalidade">
  <option value="retirada">Retirada</option>
  <option value="entrega">Entrega (+ R$ 8)</option>
</select>

<button type="submit">Calcular pedido</button>
```

Teste a navegação usando somente a tecla Tab.



```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #292524;
  background: #fff7ed;
}

main {
  width: min(42rem, 100% - 2rem);
  margin: 3rem auto;
}
```

Teste em uma janela larga e em uma janela estreita.



```
form {  
  display: grid;  
  gap: .75rem;  
  padding: 1.5rem;  
  background: white;  
  border-radius: .75rem;  
  box-shadow: 0 .5rem 1.5rem #7c2d1220;  
}  
  
input, select, button {  
  width: 100%;  
  padding: .75rem;  
  font: inherit;  
}
```

O formulário usa um fluxo vertical simples e previsível.



```
button {  
  margin-top: .5rem;  
  border: 0;  
  border-radius: .5rem;  
  color: white;  
  background: #b91c1c;  
  cursor: pointer;  
}  
  
button:hover,  
button:focus-visible {  
  background: #991b1b;  
}  
  
#resultado { display: block; margin-top: 1rem; }
```

O estado de foco torna a navegação por teclado visível.



```
const precos = {  
  calabresa: 38,  
  marguerita: 36,  
  vegetariana: 40  
};  
  
const adicionais = {  
  pequena: 0,  
  media: 8,  
  grande: 14  
};
```

Separar dados do cálculo evita uma sequência extensa de condições.



```
function calcularTotal(sabor, tamanho, quantidade, modalidade) {  
  const precoUnitario = precos[sabor] + adicionais[tamanho];  
  const taxaEntrega = modalidade === "entrega" ? 8 : 0;  
  
  return precoUnitario * quantidade + taxaEntrega;  
}
```

Essa função não conhece a página; recebe entradas e devolve uma saída.



```
const formulario = document.querySelector("#pedido");
const resultado = document.querySelector("#resultado");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  const sabor = formulario.elements.sabor.value;
  const tamanho = formulario.elements.tamanho.value;
  const quantidade = Number(formulario.elements.quantidade.value);
  const modalidade = formulario.elements.modalidade.value;
});
```

Use `console.log` para conferir os quatro valores.



```
if (!Number.isInteger(quantidade) ||  
    quantidade < 1 || quantidade > 10) {  
    resultado.textContent =  
        "Informe uma quantidade inteira entre 1 e 10.";  
    return;  
}
```

Adicione este trecho dentro do tratador de submit, após a leitura dos campos.
Teste 0, 1, 10, 11 e um campo vazio.



```
const total = calcularTotal(  
  sabor, tamanho, quantidade, modalidade  
);  
  
resultado.textContent =  
  `Total do pedido: R$ ${total.toFixed(2)}`;
```

Adicione após a validação.

Teste manualmente um caso cujo resultado você calculou antes no papel.



Caso	Entrada	Resultado esperado
Mínimo	marguerita, pequena, 1, retirada	R\$ 36,00
Tamanho	calabresa, grande, 2, retirada	R\$ 104,00
Entrega	vegetariana, média, 1, entrega	R\$ 56,00
Inválido	qualquer sabor, quantidade 0	mensagem de erro

Se o observado divergir, descubra se a falha está na entrada, regra ou apresentação.



Nesta versão, tudo executa no cliente:

- preços estão em `app.js`;
- cálculo ocorre no navegador;
- nenhum pedido é persistido;
- recarregar a página perde o estado.

Isso é adequado para aprender a interface, mas não para confirmar uma venda real.



O cliente poderia enviar apenas as escolhas:

```
{  
  "sabor": "calabresa",  
  "tamanho": "grande",  
  "quantidade": 2,  
  "modalidade": "retirada"  
}
```

O servidor consulta preços confiáveis, valida regras, calcula o total e persiste o pedido.



```
const resposta = await fetch("/api/pedidos", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(pedido)
});

if (!resposta.ok) {
  throw new Error("Não foi possível criar o pedido");
}

const pedidoCriado = await resposta.json();
```

Neste laboratório, o trecho serve para localizar a futura integração cliente-servidor.



Apresente:

- sabor escolhido;
- tamanho;
- quantidade;
- modalidade;
- total.

Exemplo:

```
2 pizzas de calabresa, tamanho grande, para retirada.  
Total: R$ 104,00
```

Não use `innerHTML` com dados digitados pelo usuário.



Requisitos:

- campo opcional cupom;
- código WEB10 oferece 10% de desconto;
- desconto incide sobre os itens, não sobre a taxa de entrega;
- código desconhecido produz mensagem clara;
- total nunca pode ser negativo.

Antes de programar, escreva três casos de teste.



Organize o código em funções:

```
function lerPedido(formulario) { /* ... */ }  
function validarPedido(pedido) { /* ... */ }  
function calcularTotal(pedido) { /* ... */ }  
function apresentarResultado(pedido, total) { /* ... */ }
```

Cada função deve ter uma responsabilidade reconhecível pelo nome.



- HTML possui estrutura semântica e `label` para cada campo;
- CSS mantém a página utilizável em telas estreitas;
- JavaScript não produz erros no console;
- entradas inválidas recebem mensagens úteis;
- quatro casos de teste foram executados;
- o grupo consegue explicar onde cada trecho executa;
- o grupo identifica o que deveria migrar para o servidor.

Parte 5 — Consolidando o modelo mental



No laboratório:

- HTML define os elementos da interface;
- CSS define a apresentação;
- JavaScript lê eventos, valida entradas e calcula;
- o navegador executa e apresenta tudo.

Ainda não existe servidor da aplicação; existe apenas um documento local ou servido estaticamente.



Quando o pedido passar a valer no mundo real:

- o servidor valida novamente as entradas;
- preços vêm de uma fonte controlada;
- regras são aplicadas de forma consistente;
- o banco registra o pedido;
- a resposta comunica sucesso ou falha;
- o cliente atualiza a interface.

Confiança e persistência mudam a fronteira da solução.



A mesma ação atravessa todas as camadas

Ao confirmar um pedido:

- 1 **apresentação:** coleta escolhas;
- 2 **cliente:** valida o formato;
- 3 **HTTP:** transporta a solicitação;
- 4 **negócio:** verifica preços e regras;
- 5 **dados:** registra o pedido;
- 6 **HTTP:** devolve o resultado;
- 7 **apresentação:** mostra a confirmação.

Essa sequência será retomada ao estudar backend e banco de dados.



Sintoma	Primeira investigação
Página sem estrutura	HTML / aba Elements
Elemento sem estilo	seletor CSS / estilos aplicados
Clique não produz efeito	Console / listener de evento
Valor calculado está errado	entradas e regra de cálculo
Requisição falhou	aba Network / status HTTP
Dado desapareceu	persistência no servidor

Comece pela evidência mais próxima do sintoma.



Síntese: quatro ideias para levar da aula

- ① Cliente e servidor são papéis em uma conversa.
- ② HTTP estrutura requisições e respostas.
- ③ Apresentação, negócio e dados têm responsabilidades diferentes.
- ④ HTML, CSS e JavaScript colaboram no navegador.

O laboratório conecta essas ideias em um fluxo observável.



Explique, sem consultar os slides:

- o que acontece após o envio do formulário;
- quais validações podem ocorrer no cliente;
- quais regras precisam ser garantidas no servidor;
- por que o banco de dados não deve ser acessado diretamente pelo navegador;
- como DevTools ajudaria a localizar uma falha.

Se você consegue narrar o percurso, já possui o modelo mental fundamental.



Transformaremos a simulação local em uma aplicação conectada:

- definiremos um recurso pedidos;
- enviaremos dados por HTTP;
- processaremos regras no servidor;
- persistiremos o resultado;
- trataremos sucesso e falha no cliente.

Ideia central

A interface inicia a conversa; a arquitetura garante que ela termine corretamente.



- MDN Web Docs. *HTML: HyperText Markup Language*.
- MDN Web Docs. *CSS: Cascading Style Sheets*.
- MDN Web Docs. *JavaScript Guide*.
- MDN Web Docs. *An overview of HTTP*.
- WHATWG. *HTML Living Standard*.
- IETF. *HTTP Semantics — RFC 9110*.